Array Problems — Java Solutions

1. Reverse the Array

Two pointer swap approach.

```
class Reverse {
    static void reverseArray(int[] arr) {
        int start = 0, end = arr.length - 1;
        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }

     public static void main(String[] args) {
         int[] arr = {1, 2, 3, 4, 5};
         reverseArray(arr);
         for (int x : arr) System.out.print(x + " ");
     }
}
```

2. Find Maximum and Minimum Element in an Array

Single pass, track min and max.

```
class MinMax {
    public static void main(String[] args) {
        int[] arr = {12, 4, 99, -3, 45, 0};
        int min = arr[0], max = arr[0];

          for (int i = 1; i < arr.length; i++) {
              if (arr[i] < min) min = arr[i];
              if (arr[i] > max) max = arr[i];
          }

          System.out.println("Min: " + min);
          System.out.println("Max: " + max);
     }
}
```

3. Find the Kth Max and Min Element of an Array

Sort and pick from both ends.

```
import java.util.Arrays;

class KthMaxMin {
    public static void main(String[] args) {
        int[] arr = {7, 10, 4, 3, 20, 15};
        int k = 3;
        Arrays.sort(arr);
```

```
        int kthMin = arr[k - 1];
        int kthMax = arr[arr.length - k];

        System.out.println("Kth Min: " + kthMin);
        System.out.println("Kth Max: " + kthMax);
    }
}
```

4. Sort an Array of 0s, 1s and 2s (Dutch National Flag)

Three pointers, single pass, no sorting algo used.

```
 class DutchFlagSort {
     static void sort012(int[] arr) {
         int low = 0, mid = 0, high = arr.length - 1;

          while (mid <= high) {
              if (arr[mid] == 0) {
                  int t = arr[low]; arr[low] = arr[mid]; arr[mid] = t;
                  low++; mid++;
              } else if (arr[mid] == 1) {
                  mid++;
              } else {
                  int t = arr[mid]; arr[mid] = arr[high]; arr[high] = t;
                  high--;
              }
          }
      }

      public static void main(String[] args) {
          int[] arr = {0, 1, 2, 0, 1, 2, 1, 0};
          sort012(arr);
          for (int x : arr) System.out.print(x + " ");
      }
 }
```

5. Move All Negative Elements to One Side of the Array

Similar to partition step of quicksort.

```
 class MoveNegatives {
     static void moveNegatives(int[] arr) {
         int j = 0;
         for (int i = 0; i < arr.length; i++) {
             if (arr[i] < 0) {
                 int t = arr[i]; arr[i] = arr[j]; arr[j] = t;
                 j++;
             }
         }
     }

      public static void main(String[] args) {
          int[] arr = {-1, 2, -3, 4, 5, -6, -7, 8};
          moveNegatives(arr);
          for (int x : arr) System.out.print(x + " ");
      }
 }
```

6. Find Union and Intersection of Two Sorted Arrays

Merge-style two pointer traversal.

```
 import java.util.ArrayList;
 import java.util.List;

 class UnionIntersection {
     public static void main(String[] args) {
         int[] a = {1, 2, 4, 5, 6};
         int[] b = {2, 3, 5, 7};

           List<Integer> union = new ArrayList<>();
           List<Integer> intersection = new ArrayList<>();
           int i = 0, j = 0;

           while (i < a.length && j < b.length) {
               if (a[i] < b[j]) {
■                    union.add(a[i]); i++;
                } else if (a[i] > b[j]) {
                    union.add(b[j]); j++;
                } else {
                    union.add(a[i]);
                    intersection.add(a[i]);
                    i++; j++;
                }
           }
           while (i < a.length) union.add(a[i++]);
           while (j < b.length) union.add(b[j++]);

           System.out.println("Union: " + union);
           System.out.println("Intersection: " + intersection);
      }
 }
```

7. Cyclically Rotate an Array by One

Store last element, shift right, place at start.

```
 class RotateByOne {
     static void rotate(int[] arr) {
         int last = arr[arr.length - 1];
         for (int i = arr.length - 1; i > 0; i--) {
             arr[i] = arr[i - 1];
         }
         arr[0] = last;
     }

      public static void main(String[] args) {
          int[] arr = {1, 2, 3, 4, 5};
          rotate(arr);
          for (int x : arr) System.out.print(x + " ");
      }
 }
```

8. Largest Sum Contiguous Subarray (Kadane's Algorithm)

Track running sum and max sum.

```
 class KadaneAlgo {
     static int maxSubArraySum(int[] arr) {
         int maxSoFar = arr[0], currMax = arr[0];

           for (int i = 1; i < arr.length; i++) {
               currMax = Math.max(arr[i], currMax + arr[i]);
               maxSoFar = Math.max(maxSoFar, currMax);
           }
           return maxSoFar;
      }

      public static void main(String[] args) {
          int[] arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
          System.out.println("Max Subarray Sum: " + maxSubArraySum(arr));
      }
 }
```

9. Minimum Number of Jumps to Reach End of an Array

Greedy: track farthest reachable index within current jump range.

```
 class MinJumps {
     static int minJumps(int[] arr) {
         int n = arr.length;
         if (n <= 1) return 0;
         if (arr[0] == 0) return -1;
■           int jumps = 1, currEnd = arr[0], farthest = arr[0];

           for (int i = 1; i < n; i++) {
               if (i == n - 1) return jumps;
               farthest = Math.max(farthest, i + arr[i]);
               if (i == currEnd) {
                   jumps++;
                   currEnd = farthest;
               }
           }
           return -1;
      }

      public static void main(String[] args) {
          int[] arr = {1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9};
          System.out.println("Min Jumps: " + minJumps(arr));
      }
 }
```

10. Find Duplicate in an Array of N+1 Integers

Floyd's cycle detection (treat array as linked list).

```
 class FindDuplicate {
     static int findDuplicate(int[] arr) {
         int slow = arr[0], fast = arr[0];

           do {
               slow = arr[slow];
```

```
            fast = arr[arr[fast]];
        } while (slow != fast);

        slow = arr[0];
        while (slow != fast) {
            slow = arr[slow];
            fast = arr[fast];
        }
        return slow;
    }

    public static void main(String[] args) {
        int[] arr = {1, 3, 4, 2, 2};
        System.out.println("Duplicate: " + findDuplicate(arr));
    }
}
```

11. Merge 2 Sorted Arrays Without Using Extra Space

```
Gap method (Shell-sort style) for O(1) extra space.
class MergeWithoutExtraSpace {
    static void merge(int[] a, int[] b) {
        int n = a.length, m = b.length;
        int gap = (n + m + 1) / 2;

        while (gap > 0) {
            int i = 0, j = gap;
            while (j < n + m) {
                int val1 = (i < n) ? a[i] : b[i - n];
                int val2 = (j < n) ? a[j] : b[j - n];

                if (val1 > val2) {
                    if (i < n && j < n) {
                        int t = a[i]; a[i] = a[j]; a[j] = t;
                    } else if (i < n && j >= n) {
                        int t = a[i]; a[i] = b[j - n]; b[j - n] = t;
                    } else {
                        int t = b[i - n]; b[i - n] = b[j - n]; b[j - n] = t;
■                   }
                }
                i++; j++;
            }
            gap = (gap == 1) ? 0 : (gap + 1) / 2;
        }
    }

    public static void main(String[] args) {
        int[] a = {1, 5, 9, 10, 15, 20};
        int[] b = {2, 3, 8, 13};
        merge(a, b);
        for (int x : a) System.out.print(x + " ");
        for (int x : b) System.out.print(x + " ");
    }
}
```

12. Best Time to Buy and Sell Stock

Track minimum price seen so far, update max profit.

```
class BestTimeToBuySell {
    static int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE, maxProfit = 0;

          for (int price : prices) {
              if (price < minPrice) minPrice = price;
              else if (price - minPrice > maxProfit) maxProfit = price - minPrice;
          }
          return maxProfit;
     }

     public static void main(String[] args) {
         int[] prices = {7, 1, 5, 3, 6, 4};
         System.out.println("Max Profit: " + maxProfit(prices));
     }
}
```

13. Find All Pairs in an Integer Array Whose Sum is Equal to Given Number

HashSet based single pass.

```
import java.util.HashSet;

class PairSum {
    static void findPairs(int[] arr, int target) {
        HashSet<Integer> seen = new HashSet<>();

          for (int num : arr) {
              int complement = target - num;
              if (seen.contains(complement)) {
                  System.out.println("(" + complement + ", " + num + ")");
              }
              seen.add(num);
          }
     }

     public static void main(String[] args) {
         int[] arr = {1, 5, 7, -1, 5};
         findPairs(arr, 6);
     }
}
```

14. Find Common Elements in 3 Sorted Arrays

Three pointer merge traversal.

```
■import java.util.ArrayList;
import java.util.List;

class CommonInThreeArrays {
    static List<Integer> commonElements(int[] a, int[] b, int[] c) {
        List<Integer> result = new ArrayList<>();
```

```
        int i = 0, j = 0, k = 0;

          while (i < a.length && j < b.length && k < c.length) {
              if (a[i] == b[j] && b[j] == c[k]) {
                  result.add(a[i]);
                  i++; j++; k++;
              } else if (a[i] < b[j]) {
                  i++;
              } else if (b[j] < c[k]) {
                  j++;
              } else {
                  k++;
              }
          }
          return result;
      }

      public static void main(String[] args) {
          int[] a = {1, 5, 10, 20, 40, 80};
          int[] b = {6, 7, 20, 80, 100};
          int[] c = {3, 4, 15, 20, 30, 70, 80, 120};
          System.out.println(commonElements(a, b, c));
      }
}
```

15. Rearrange Array in Alternating Positive and Negative Items

Right rotation approach with O(1) extra space.

```
class RearrangeAltPosNeg {
    static void rightRotate(int[] arr, int start, int end) {
        int temp = arr[end];
        for (int i = end; i > start; i--) arr[i] = arr[i - 1];
        arr[start] = temp;
    }

      static void rearrange(int[] arr) {
          int outOfPlace = -1;
          for (int i = 0; i < arr.length; i++) {
              if (outOfPlace >= 0) {
                  if ((arr[i] >= 0 && arr[outOfPlace] < 0) ||
                      (arr[i] < 0 && arr[outOfPlace] >= 0)) {
                      rightRotate(arr, outOfPlace, i);
                      outOfPlace = (i - outOfPlace > 2) ? outOfPlace + 2 : -1;
                  }
              }
              if (outOfPlace == -1 &&
                  ((arr[i] >= 0 && i % 2 == 0) || (arr[i] < 0 && i % 2 != 0))) {
                  outOfPlace = i;
              }
          }
      }

      public static void main(String[] args) {
          int[] arr = {1, 2, 3, -4, -1, 4};
          rearrange(arr);
          for (int x : arr) System.out.print(x + " ");
```

```
    }
}
```

16. Find if There is Any Subarray with Sum Equal to 0

■HashSet of prefix sums.

```
import java.util.HashSet;

class ZeroSumSubarray {
    static boolean hasZeroSumSubarray(int[] arr) {
        HashSet<Integer> prefixSums = new HashSet<>();
        int sum = 0;

          for (int num : arr) {
              sum += num;
              if (sum == 0 || prefixSums.contains(sum)) return true;
              prefixSums.add(sum);
          }
          return false;
     }

     public static void main(String[] args) {
         int[] arr = {4, 2, -3, 1, 6};
         System.out.println("Has zero sum subarray: " + hasZeroSumSubarray(arr));
     }
}
```

17. Find Factorial of a Large Number

Array-based multiplication (big integer simulation).

```
class LargeFactorial {
    static void multiply(int[] result, int x, int[] size) {
        int carry = 0;
        for (int i = 0; i < size[0]; i++) {
            int prod = result[i] * x + carry;
            result[i] = prod % 10;
            carry = prod / 10;
        }
        while (carry != 0) {
            result[size[0]] = carry % 10;
            carry /= 10;
            size[0]++;
        }
    }

     static void factorial(int n) {
         int[] result = new int[500];
         result[0] = 1;
         int[] size = {1};

          for (int x = 2; x <= n; x++) multiply(result, x, size);

          StringBuilder sb = new StringBuilder();
          for (int i = size[0] - 1; i >= 0; i--) sb.append(result[i]);
```

```
        System.out.println(n + "! = " + sb);
    }

    public static void main(String[] args) {
        factorial(50);
    }
}
```

18. Find Maximum Product Subarray

Track running max and min (handles negative flips).

```
class MaxProductSubarray {
    static int maxProduct(int[] arr) {
        int maxProd = arr[0], minProd = arr[0], result = arr[0];

        for (int i = 1; i < arr.length; i++) {
■           int num = arr[i];
            if (num < 0) {
                int temp = maxProd;
                maxProd = minProd;
                minProd = temp;
            }
            maxProd = Math.max(num, maxProd * num);
            minProd = Math.min(num, minProd * num);
            result = Math.max(result, maxProd);
        }
        return result;
    }

    public static void main(String[] args) {
        int[] arr = {2, 3, -2, 4};
        System.out.println("Max Product Subarray: " + maxProduct(arr));
    }
}
```

19. Find Longest Consecutive Subsequence

HashSet based, start counting only from sequence beginnings.

```
import java.util.HashSet;

class LongestConsecutiveSubsequence {
    static int longestConsecutive(int[] arr) {
        HashSet<Integer> set = new HashSet<>();
        for (int num : arr) set.add(num);

        int longest = 0;
        for (int num : set) {
            if (!set.contains(num - 1)) {
                int length = 1;
                while (set.contains(num + length)) length++;
                longest = Math.max(longest, length);
            }
        }
        return longest;
```

```
    }

    public static void main(String[] args) {
        int[] arr = {100, 4, 200, 1, 3, 2};
        System.out.println("Longest Consecutive Sequence: " + longestConsecutive(arr));
    }
}
```

20. Find the Triplet That Sums to a Given Value

Sort array, then two pointer for each fixed element.

```
import java.util.Arrays;

class TripletSum {
    static boolean findTriplet(int[] arr, int target) {
        Arrays.sort(arr);
        int n = arr.length;

          for (int i = 0; i < n - 2; i++) {
              int left = i + 1, right = n - 1;
              while (left < right) {
                  int sum = arr[i] + arr[left] + arr[right];
                  if (sum == target) {
                      System.out.println(arr[i] + ", " + arr[left] + ", " + arr[right]);
                      return true;
                  } else if (sum < target) {
                      left++;
                  } else {
■                     right--;
                }
            }
        }
        return false;
    }

    public static void main(String[] args) {
        int[] arr = {12, 3, 4, 1, 6, 9};
        int target = 24;
        if (!findTriplet(arr, target)) System.out.println("No triplet found");
    }
}
■
```